

TMX-080 (H1, STATE-HOOK-RACE-001) — Fresh Investigation Evidence, 2026-09-29

Governing instruction

Operator instruction: investigate item 3 (the highest-risk, twice-reverted reopened item) with the SAME rigor as items 1 and 2 — validated and verified fully deterministically on the live installed latest codebase — before any release. Item carries two prior fix attempts, both reverted, one of which made the failure rate WORSE (5/5 reproducible, up from ~20-25% intermittent). Per the Iron Law, no third fix attempt without a confirmed root cause.

Stream A — forensics on the two 2026-08-13 fix attempts

- Method: `git log --all --grep="TMX-080|H1|STATE-HOOK-RACE" -i, git log --all --since=2026-08-08 --until=2026-08-16 -- scripts/tests/27_state_persistence.sh scripts/tmx.template scripts/tmx-recycler.sh, git fsck --no-reflogs --unreachable --dangling, git reflog show --all.`
- Finding: only two real commits bracket this item — 84d5201 (2026-08-10, item creation) and f99a9de (2026-08-13, doc-only investigation writeup). Zero commits touch any of the three implicated files in the Aug 8-16 window. 65 dangling commits exist repo-wide but are ALL dated 2026-09-01/02 (unrelated stash residue from a separate, later investigation) — none reference TMX-080/H1/STATE-HOOK-RACE, none are dated August 2026.
- Conclusion: BOTH fix attempts and BOTH reverts happened entirely in the uncommitted working tree, between the two doc-only commits. Neither was ever committed at any point. The current tree is byte-for-byte identical to the pre-both-attempts state for all three implicated files.

- Attempt 1: delayed `systemctl --user stop "$SCOPE_UNIT"` (in `scripts/tmx.template's kill-session verb` + an analogous change in `scripts/tmx-recycler.sh`) until a new `_wait_scope_quiescent` helper observed the scope's cgroup as empty. Recorded outcome: made the failure WORSE (5/5 reproducible, always iteration 1 — matching the ORIGINAL, already-disproven diagnosis pattern, not the genuinely-intermittent iteration-2/3 pattern this SAME investigation had just proven with 8 fresh runs). No causal hypothesis is recorded for why it got worse.
- Attempt 2 ("corrected"): resolved the scope's cgroup path ONCE instead of re-querying `systemctl show's ControlGroup` field on every poll iteration (which itself has a race — `ControlGroup` reports empty almost immediately once the tracked main PID exits, before a still-running child has necessarily finished). Also failed to resolve the real test — DESPITE "both fix attempts working correctly in isolated, simplified manual reproductions." The manual repro used an artificially-slowed hook command (`sleep 0.05; tmx-state-bin record ...`) — a hand-constructed simplification, never a driven replay of the real test's exact sequence (`cd -> poll-for-landing -> run-shell record -> kill-session -> poll-for-recall`, inside the full harness). This exact-reproduction- sequence gap is the likely reason both attempts looked correct in isolation while leaving the real test still failing.

Stream B — mechanism deep-dive, #{pane_current_path} / tcgetpgrp

Source read directly from this project's pinned tmux 3.7b checkout.

`tmux/osdep-linux.c:29-61 (osdep_get_name)` and `:63-89 (osdep_get_cwd)`: both call `tcgetpgrp(fd)` on the PANE'S OWN pty master fd (`wp->fd`, confirmed `spawn.c:391`), then use the returned `pgrp` value DIRECTLY as a PID against `/proc/<pgrp>/cmdline` OR `/proc/<pgrp>/cwd`. No caching — every call performs a fresh `tcgetpgrp()` + `/proc` read. `osdep_get_cwd` has a session-id fallback on primary-read failure; `osdep_get_name` does not.

Call sites: `tmux/format.c:886-903 (format_cb_current_command, backs #{pane_current_command})` and `:911-923 (format_cb_current_path, backs #{pane_current_path})` — both call the `osdep` functions directly on `wp->fd`, no intermediate sanity check.

scripts/tests/27_state_persistence.sh confirmed to query `{pane_current_path}` via exactly this path THREE times per iteration: line 121 (Phase 2, right after `tmx new`), line 164 (Phase 3, `cd-landing poll`), line 174 (the hook's own `run-shell` command line). Confirmed (via `cmd-run-shell.c:136-141 / cmd-display-message.c:128-129`) that all three resolve synchronously, at command-processing time, against the same `target->wp` pane context — none is insulated from the mechanism. Also confirmed `run-shell`'s own spawned job forks its OWN separate `pty` (`job.c:116`), distinct from `wp->fd` — ruling out "the hook's own child is the confound."

External research (WebSearch/WebFetch, sources cited):

- General mechanism (tmux intentionally reports whatever is currently the foreground process group, by design): well corroborated.
- [tmux/tmux#1889](#) — a live, open report of stale `{pane_current_path}` reads, confirming the SYMPTOM CLASS is real and previously reported generically; comment thread did not fetch, so it does NOT corroborate this specific mechanism as #1889's cause (unconfirmed either way).
- [microsoft/WSL#1063](#) — independently corroborates `tcgetpgrp(fd)` on the `pty MASTER fd` is a known-fragile construct in the wild (a different failure shape: hard -1 failure on WSL, not staleness) — general corroboration only.
- [bash 5.1 patch bash51-003](#) — bash's own official patch notes confirm command-substitution subprocesses genuinely interact with process-group placement (narrower scope than "every \$ (...)", full diff not fetched).
- No external source names "oh-my-bash / shell-framework subprocess racing tmux's `tcgetpgrp`-based resolution" specifically — this is original analysis to this project's own investigation, not a previously-documented bug pattern.

Narrowing check performed independently (this session, read-only, on this host): the active oh-my-bash theme (`font, ~/.oh-my-bash/themes/font/font.theme.sh`) registers `_omb_theme_PROMPT_COMMAND` into bash's native `PROMPT_COMMAND` via `_omb_util_add_prompt_command` (`~/.oh-my-bash/lib/utls.sh`) — this runs on EVERY prompt draw, not only at shell startup. `_omb_theme_PROMPT_COMMAND` calls `$(scm_prompt_char_info)`, which (per `~/.oh-my-bash/lib/omb-prompt-base.sh`) forks real `git` subprocesses via `$(...):scm()` calls `_omb_prompt_git rev-parse --is-inside-work-tree`; with `SCM_GIT_SHOW_MINIMAL_INFO=true` (set by this theme), `git_prompt_minimal_info()` additionally calls `git_clean_branch` (`symbolic-ref -q HEAD`), a `rev-parse --short HEAD` fallback, and `git`

status --porcelain when inside a git worktree. Confirmed: this per-prompt git-forking mechanism genuinely exists and fires on this exact host, on the exact code path (every prompt draw, including the one right after the test's `cd ... Enter`).

HOWEVER — important correction surfaced by the live-reproduction agent (Stream C): bash command-substitution subshells (`$(...)`) do not, under standard bash job-control semantics, receive a `tcsetpgrp()` foreground handoff the way a directly-launched foreground pipeline does — a command-substitution subshell typically inherits the parent's EXISTING process group rather than being granted a new one and handed the terminal. So confirming the git forks exist does not, by itself, confirm they are mechanically capable of becoming the pane's foreground process group the way the candidate hypothesis requires. This narrows confidence in the SPECIFIC per-prompt-git-fork mechanism without ruling it out entirely (other subprocess-launch shapes in the shell's startup/session lifecycle could still be eligible under normal job control).

Stream C — fresh live reproduction, today's build

`bash scripts/setup.sh --rebuild` succeeded on the first attempt. Fresh artifacts confirmed present (`scripts/tmx`, `scripts/tmx-state-bin`, `tmux/build/bin/tmux`, all timestamped today).

10 standalone invocations of `scripts/tests/27_state_persistence.sh` (30 internal iterations, 3 per invocation per the test's own \$11.450 reliability loop) + 2 additional invocations run under live foreground-pgrp sampling (6 more internal iterations) + 1 independent pass captured from the rebuild's own `run_all.sh` sweep = **13 independent invocations, 39 internal iterations, 0 failures (0%)**.

2026-08-13 baseline: $2/8 \approx 25\%$ (failures on iterations 2 and 3, never iteration 1). Today: 0/10 direct runs, corroborated by 39/39 total iterations across every independent invocation this session — a large, methodologically-consistent drop.

Live foreground-process-group observation (captured twice independently, via a busy-wait on the test's socket file appearing followed by a tight-loop `ps -o pid,pgid,ppid,stat,comm -t <pane_tty>` sampler, no forks in the busy-wait itself):

- Run 1: 110 samples across ~2.9s (all 3 iterations, including pane-recreate churn between iterations).
- Run 2: 107 samples, same shape.
- Every single populated sample in both runs showed exactly one foreground process: `bash`, state `Ss+` (session leader, sleeping, foreground), `PGID == PID`. No other command name (no `git`, no oh-my-bash helper) ever appeared as foreground in any sample, in either run, including the windows immediately following each `send-keys "cd ..."` and around the `run-shell` hook fire.
- Honest limitation: sampling was `ps`-fork-overhead-bound at roughly 10-60ms per sample; a process-group reassignment lasting only a few ms could in principle fall entirely between two samples and go undetected. The test emits no per-phase timestamps, so exact correlation between sample times and the script's internal phase boundaries could not be proven for every iteration.

Disposition

Neither candidate mechanism (the `KillMode=control-group` kill-session race, nor the `tcgetpgrp` pane-state race) is confirmed against a live failure, because none occurred anywhere in this cycle's extensive fresh testing to study. No third fix attempt was made. Existing test coverage (condition-based polling, unchanged since 2026-06-28, never touched by either reverted attempt) is confirmed as the correct standing mitigation. Full disposition and status recorded in `Issues.md` §H1 "Investigation update (2026-09-29)" and `CONTINUATION.md` §3.39.